DBMS

GATE CSE 2027 · complete formula, rule, method & trap sheet

103 reference boxes · **8** chapters ·

ER Model | Relational Model, Algebra & Keys | SQL | Integrity Constraints & Normal Forms | File Organization & Indexing (B/B+ Trees) | Transactions & Concurrency Control | Problem → Standard Method Playbook | Last-minute one-liners

1 ER Model

The building blocks

REFERENCE TABLE

Concept	Meaning	Example
Entity	A distinguishable "thing" you want to store data about	Student, Course
Attribute	A property of an entity	Student's name, roll number
Relationship	An association between two or more entities	Student ENROLLS_IN Course

Types of attributes

REFERENCE TABLE

Type	Meaning
Simple	Cannot be divided further (e.g. Age)
Composite	Made of sub-parts (e.g. Address = Street+City+Pincode)
Single-valued	One value per entity (e.g. DOB)
Multi-valued	Can have many values (e.g. Phone Numbers) — shown with a double oval
Derived	Computed from another attribute (e.g. Age derived from DOB) — shown with a dashed oval

Cardinality ratios & participation constraints

KEY POINT

Cardinality ratio — how many instances of one entity can relate to instances of another: **1:1**, **1:N**, or **M:N**.

KEY POINT

Participation constraint — **total** (every entity instance MUST participate, drawn as a double line) vs **partial** (participation is optional, single line).

KEY POINT

Weak entity: an entity with no primary key of its own — it's identified only via a relationship to an "owner" (identifying) entity plus a **partial key**. Drawn with a double-bordered rectangle, connected via a double-bordered diamond (identifying relationship).

Mapping an ER diagram to relational tables — the GATE trick

REFERENCE TABLE

ER construct	How many tables needed
Strong entity	1 table (attributes become columns, key becomes primary key)
1:1 relationship	Can often be merged into ONE of the two entity tables (foreign key on either side, or the side with total participation)
1:N relationship	Add the "1" side's primary key as a foreign key in the "N" side's table — no separate table needed
M:N relationship	ALWAYS needs its own separate table, holding both entities' primary keys (as a composite key) plus any relationship attributes
Weak entity	1 table, with the owner's primary key + the weak entity's partial key together forming its primary key

FORMULA / RULE

Worked numerical

Q: An ER diagram has: Entity A (strong), Entity B (strong), Entity C (weak, owned by B). Relationships: A–B is 1:N, B–C is the identifying relationship (1:N, total participation of C). Find the MINIMUM number of tables needed.

A:

```
Entity A → 1 table
Entity B → 1 table (also absorbs the 1:N relationship from A, via a foreign key A_id in B)
Entity C (weak) → 1 table (primary key = B_id + C's partial key; no separate relationship table needed since it's identifying)

Total = 3 tables
```

The golden rule: **only M:N relationships force a genuinely new, separate table** — every 1:1 and 1:N relationship can be absorbed as a foreign key into an existing entity's table.

Extended ER (EER) — generalization, specialization, aggregation

KEY POINT

Specialization is the **top-down** process of defining sub-classes of an entity set (Person → Student, Employee); **generalization** is the reverse **bottom-up** process of merging similar entity sets into one super-class. Sub-classes **inherit** all attributes and relationships of the super-class (the "IS-A" relationship). **Aggregation** treats a whole relationship (plus its participants) as a single higher-level entity so it can itself take part in another relationship.

REFERENCE TABLE

Constraint on a specialization	Meaning
Disjoint (d)	An entity can belong to **at most one** sub-class
Overlapping (o)	An entity may belong to **several** sub-classes at once
Total (double line)	Every super-class entity must belong to **at least one** sub-class
Partial (single line)	A super-class entity may belong to no sub-class at all

KEY POINT

Mapping EER to tables (4 standard options)

```
1. One table per class:   Super(K, a…)  +  Sub_i(K, own attrs…)   — works for every constraint combination
2. One table per sub-class only (each holds inherited + own attrs) — needs total + disjoint
3. Single table with a type flag (all attrs, NULLs for unused) — best for disjoint
4. Single table with one boolean flag per sub-class — handles overlapping
```

2 Relational Model, Algebra & Keys

Basic terminology

REFERENCE TABLE

Term	Meaning
Relation	A table
Tuple	A row
Attribute	A column
Degree	Number of attributes (columns)
Cardinality	Number of tuples (rows)

Keys

REFERENCE TABLE

Key type	Meaning
Super key	Any set of attributes that uniquely identifies a tuple (may include extra, unnecessary attributes)
Candidate key	A MINIMAL super key — remove any attribute and it stops being unique
Primary key	The candidate key chosen by the designer to be THE identifier
Foreign key	An attribute in one table that references the primary key of another table

GATE FAVOURITE

GATE angle: counting keys via Functional Dependencies

Given a relation R and a set of functional dependencies (FDs), a **candidate key** is a minimal attribute set whose **closure** (under those FDs) equals ALL attributes of R.

Attribute closure algorithm: start with the given set, repeatedly add the RHS of any FD whose LHS is already fully contained in your current set, until nothing new is added.

FORMULA / RULE

Worked numerical

Q: R(A,B,C,D) with FDs: A→B, B→C, C→D. Find a candidate key.

A:

```
Try {A}: closure({A}) → A,B (from A→B) → A,B,C (from B→C) → A,B,C,D (from C→D)
closure({A}) = {A,B,C,D} = all attributes → {A} is a candidate key (and it's minimal, being just 1 attribute)
```

Relational Algebra operators

REFERENCE TABLE

Operator	Symbol	Meaning
Selection	$\sigma_{condition}(R)$	Filter ROWS matching a condition
Projection	$\pi_{attributes}(R)$	Keep only specific COLUMNS (duplicates removed)
Union	$R \cup S$	Rows in R or S (same schema required)
Set difference	$R - S$	Rows in R but not in S
Cartesian product	$R \times S$	Every row of R paired with every row of S
Rename	ρ	Rename a relation or its attributes
Natural Join	$R \bowtie S$	Join on all common attribute names, keep one copy of each

WORKED EXAMPLE

Worked numerical

Q: R has 5 tuples over attributes (A,B); S has 3 tuples over attributes (B,C). If B has 2 common values between R and S, and matching creates exactly 6 combined tuples, how many tuples are in R ⋈ S (natural join), vs R × S (Cartesian product)?

A:

```
R × S (Cartesian product) = 5 × 3 = 15 tuples (every R row paired with every S row, ignoring B values)
R ⋈ S (natural join) = 6 tuples (only pairs where R.B = S.B actually match, as given)
```

Natural join is always ≤ the Cartesian product size — it's effectively "Cartesian product, then filter where the shared attribute matches, then drop the duplicate column".

WORKED EXAMPLE

Worked numerical — combining operators

Q: Given Employee(eid, name, dept, salary), write a relational algebra expression for "names of employees in the 'Sales' department earning more than 50000", and identify which operators are used.

A:

```
π_name ( σ_(dept='Sales' AND salary>50000) (Employee) )
```

Uses **Selection** (σ) to filter rows matching the department+salary condition, then **Projection** (π) to keep only the name column. This "select-then-project" pattern is the relational-algebra equivalent of a SQL `SELECT name FROM Employee WHERE dept='Sales' AND salary>50000`.

Join variants

REFERENCE TABLE

Join	Meaning
Theta join $R \bowtie_\theta S$	$\sigma_\theta(R \times S)$ for any condition θ (<, >, =, ≠…)
Equi-join	Theta join where θ uses only `=` — both copies of the join column are kept
Natural join	Equi-join on ALL common attribute names, keeping one copy of each
Left / Right / Full outer join	Natural (or θ) join PLUS the unmatched tuples of the left / right / both sides, padded with NULLs

Division (÷) — the "for ALL" operator

MUST REMEMBER

Definition. For R(A, B) and S(B), `R ÷ S` returns every value **a** (over the attributes of R that are NOT in S) such that the pair **(a, b) appears in R for EVERY b in S**.

```
R ÷ S  =  π_A(R)  −  π_A( (π_A(R) × S) − R )
          ──────     ──────────────────────
          all candidates     candidates that are MISSING at least one required b → disqualified
```

Result schema = attrs(R) − attrs(S). If S is empty, R ÷ S = π_A(R). Division is the relational-algebra answer to every "…*all* / *every*…" query (students who took *all* required courses, suppliers who supply *all* parts).

WORKED EXAMPLE

Worked numerical — division

Q: Enrolled(student, course) = {(Amy,DB),(Amy,OS),(Amy,CN),(Ben,DB),(Ben,OS),(Cara,DB),(Cara,CN)}; Required(course) = {DB, OS}. Compute Enrolled ÷ Required.

A:

```
Candidates π_student(Enrolled) = {Amy, Ben, Cara}
Candidates × Required = {Amy,Ben,Cara} × {DB,OS} = 6 pairs
Missing pairs = (those 6) − Enrolled = {(Cara, OS)}   → Cara is disqualified
Result = {Amy, Ben, Cara} − {Cara} = {Amy, Ben}
```

Same query in SQL needs a double negation: `SELECT DISTINCT e.student FROM Enrolled e WHERE NOT EXISTS (SELECT * FROM Required r WHERE NOT EXISTS (SELECT * FROM Enrolled x WHERE x.student=e.student AND x.course=r.course))` — "there is no required course that this student has NOT taken".

GATE FAVOURITE

GATE trap

Division is **not** "Cartesian product in reverse". Extra courses (Amy's CN) don't matter — only whether ALL of S is covered. Also |R ÷ S| ≤ |π_A(R)| always.

Tuple Relational Calculus (TRC) — a quick contrast

NOTES

Where relational algebra is **procedural** (specifies HOW to compute the result, step by step), Tuple Relational Calculus is **declarative** (specifies WHAT you want, using predicate logic over tuple variables) — e.g. `{ t | Employee(t) ∧ t.dept='Sales' ∧ t.salary>50000 }` describes the same query as above, using logical implication/conjunction instead of chained operators. Both are proven to be equally expressive (relationally complete).

3 SQL

Basic query anatomy

KEY POINT

```
SELECT [columns]
FROM [table(s)]
WHERE [row filter — applied BEFORE grouping]
GROUP BY [columns]
HAVING [group filter — applied AFTER grouping]
ORDER BY [columns]
```

GATE FAVOURITE

GATE trap

`WHERE` filters individual rows before any grouping happens; `HAVING` filters entire GROUPS after aggregation. You cannot use an aggregate function (like `COUNT(*)`) inside a `WHERE` clause — it must go in `HAVING`.

Joins — with example tables

KEY POINT

```
Student            Enroll
sid | name          sid | course
1   | Amy           1   | CS101
2   | Ravi          1   | CS102
3   | Zara          4   | CS101   (sid=4 doesn't exist in Student!)
```

REFERENCE TABLE

Join type	Result on the tables above
INNER JOIN (Student ⋈ Enroll on sid)	3 rows: (Amy,CS101), (Amy,CS102), and nothing for Ravi/Zara (no match), sid=4 row dropped (no matching student)
LEFT OUTER JOIN (Student left, Enroll right)	Amy's 2 rows + Ravi(NULL) + Zara(NULL) = 4 rows — every Student row kept even without a match
RIGHT OUTER JOIN	Amy's 2 rows + (NULL, CS101) for sid=4 = 3 rows — every Enroll row kept even without a match
FULL OUTER JOIN	All of the above combined = 5 rows (Amy×2, Ravi-NULL, Zara-NULL, NULL-sid4)

Aggregate functions

FORMULA / RULE

Worked numerical

Q: Employee(eid, dept, salary): (1,'CS',40000), (2,'CS',60000), (3,'EE',50000), (4,'EE', NULL). What does `SELECT dept, COUNT(*), COUNT(salary), AVG(salary) FROM Employee GROUP BY dept;` return?

A:

```
dept | COUNT(*) | COUNT(salary) | AVG(salary)
CS   |    2     |      2        |   50000        (avg of 40000,60000)
EE   |    2     |      1        |   50000        (avg ignores the NULL, uses only 50000)
```

Key trap: `COUNT(*)` counts ALL rows including NULLs; `COUNT(column)` and every other aggregate (`SUM` , `AVG` , `MIN` , `MAX`) silently **ignore NULL values** in that column.

Nested queries

KEY POINT

IN / NOT IN

`WHERE dept IN (SELECT dept FROM ... )` — checks membership in a subquery's result set.

KEY POINT

EXISTS / NOT EXISTS

`WHERE EXISTS (SELECT * FROM ... WHERE ...)` — checks if a correlated subquery returns ANY row at all.

COMMON TRAP

The classic NULL + NOT IN trap

If the subquery's result set for a `NOT IN` contains even ONE `NULL` , the ENTIRE outer query returns **zero rows** — because comparing anything to NULL yields UNKNOWN, not TRUE/FALSE, and `NOT IN` needs every comparison to definitively be "not equal". GATE loves testing this exact gotcha.

Set operations

REFERENCE TABLE

Operator	Meaning
UNION	Rows in either query (duplicates removed)
UNION ALL	Rows in either query (duplicates KEPT)
INTERSECT	Rows in BOTH queries
EXCEPT / MINUS	Rows in the first query but NOT in the second

DDL — defining and changing schema

REFERENCE TABLE

Command	Effect
`CREATE TABLE`	Define columns, types, `PRIMARY KEY`, `UNIQUE`, `NOT NULL`, `CHECK`, `FOREIGN KEY`
`ALTER TABLE`	Add / drop / modify a column or constraint
`DROP TABLE`	Removes the table **structure and data** completely
`TRUNCATE TABLE`	Removes **all rows**, keeps the structure; no WHERE clause
`DELETE FROM … WHERE`	(DML) removes the selected rows, structure stays

KEY POINT

Referential actions on a foreign key: `ON DELETE / ON UPDATE` + `NO ACTION / RESTRICT` (reject the change — the default), `CASCADE` (propagate the delete/update to child rows), `SET NULL`, `SET DEFAULT`.

```
CREATE TABLE Emp (
  eid INT PRIMARY KEY,
  did INT REFERENCES Dept(did) ON DELETE CASCADE,
  salary INT CHECK (salary > 0)
);
```

Views

NOTES

A **view** is a stored query — a *virtual table* whose rows are computed from the base tables each time it is used: `CREATE VIEW HighPaid AS SELECT eid, salary FROM Emp WHERE salary > 50000;` Views give security (expose only some columns/rows) and simplify queries.

KEY POINT

Updatable views (INSERT/UPDATE/DELETE pass through to the base table) generally need: a **single base table**, no `DISTINCT`, no aggregates, no `GROUP BY` / `HAVING`, and the view must include the base table's key / NOT NULL columns. `WITH CHECK OPTION` rejects any update through the view that would make the row disappear from the view. A **materialized view** (unlike a plain view) physically stores its result and must be refreshed.

Other SQL features worth recognising

REFERENCE TABLE

Feature	What it does
Trigger	Event–Condition–Action rule that runs automatically on INSERT / UPDATE / DELETE, e.g. `CREATE TRIGGER t AFTER UPDATE ON emp FOR EACH ROW …`; used for audit logs and derived data
Cursor / embedded SQL	Lets a host-language program fetch a result set one row at a time (DECLARE, OPEN, FETCH, CLOSE)
Stored procedure	Named SQL program kept and run inside the server, which cuts network round trips
GRANT / REVOKE	Access control: `GRANT SELECT ON emp TO alice`; `REVOKE` withdraws it; `WITH GRANT OPTION` lets the user pass it on
COMMIT / ROLLBACK / SAVEPOINT	Transaction control: make changes permanent, undo all of them, or undo back to a marked point

4 Integrity Constraints & Normal Forms

Integrity constraints

REFERENCE TABLE

Constraint	Meaning
Domain constraint	A column's value must come from its declared data type/domain
Entity integrity	Primary key columns can NEVER be NULL
Referential integrity	A foreign key must either be NULL or match an existing primary key value in the referenced table
NOT NULL / UNIQUE / CHECK	Column-level rules enforced on every insert/update

Functional Dependencies & Armstrong's Axioms

NOTES

An FD `X → Y` means: whenever two tuples agree on X, they must also agree on Y. It's **trivial** if $Y \subseteq X$ (always true, tells you nothing new); otherwise **non-trivial**.

KEY POINT

Armstrong's Axioms (sound and complete — derive every valid FD from these):

```
Reflexivity:    if Y ⊆ X, then X → Y              (trivial FDs)
Augmentation:   if X → Y, then XZ → YZ             (add same attrs to both sides)
Transitivity:   if X → Y and Y → Z, then X → Z
```

Derived rules (provable from the three above, but useful shortcuts): **Union** ($X \to Y, X \to Z \Longrightarrow X \to YZ$), **Decomposition** ($X \to YZ \Longrightarrow X \to Y$ and $X \to Z$), **Pseudotransitivity** ($X \to Y, WY \to Z \Longrightarrow WX \to Z$).

Minimal (canonical) cover of an FD set

NOTES

A **minimal cover** F_c of F is an equivalent FD set (same closure F^+) in which (1) every RHS is a **single attribute**, (2) no LHS has an **extraneous attribute**, and (3) no FD is **redundant**. It is the starting point of the **3NF synthesis** algorithm (one relation per FD, then add a candidate key if none is present, then drop subsumed relations).

KEY POINT

Algorithm

```
Step 1  Split RHS:                X→YZ  becomes  X→Y, X→Z
Step 2  Remove extraneous LHS:    in XA→B, A is extraneous if
B ∈ closure(X) under F
Step 3  Remove redundant FDs:     drop X→Y if Y ∈ closure(X)
using the OTHER FDs
```

Do the steps **in this order**. The minimal cover is **not unique** — different orders can give different (but equivalent) covers.

WORKED EXAMPLE

Worked numerical

Q: F = { A→BC, B→C, A→B, AB→C }. Find a minimal cover.

A:

```
Step 1: A→B, A→C, B→C, A→B (duplicate), AB→C
Step 2: AB→C — is A extraneous? closure({B}) = {B,C}
contains C → yes, A is extraneous → AB→C becomes B→C
(duplicate)
        F = { A→B, A→C, B→C }
Step 3: A→C — drop it and compute closure({A}) with {A→B,
B→C} = {A,B,C} ∋ C → redundant, remove
Minimal cover = { A→B, B→C }
```

Normal forms — the ladder

REFERENCE TABLE

Form	Requirement
1NF	Every attribute holds only atomic (indivisible), single values — no repeating groups or multi-valued columns
2NF	1NF + no **partial dependency** (no non-key attribute depends on only PART of a composite candidate key)
3NF	2NF + no **transitive dependency** (no non-key attribute depends on another non-key attribute)
BCNF	For every non-trivial FD X→Y, X must be a **super key** (stricter than 3NF — no exceptions allowed)

FORMULA / RULE

Worked numerical — finding the highest normal form

Q: R(A,B,C,D) with candidate key {A,B} and FDs: AB→CD, C→D. What is the highest normal form R satisfies?

A:

```
Check 1NF: assume all atomic values → ✓ 1NF holds

Check 2NF: is there a partial dependency? Candidate key is
{A,B} (composite).
  Does C or D depend on just A, or just B alone (not both)?
  We're only given AB→CD and C→D — no dependency on A alone
or B alone → no partial dependency → ✓ 2NF holds

Check 3NF: is there a transitive dependency (non-key → non-
key)?
  C→D: here C is a NON-KEY attribute, and D is also non-key
→ this IS a transitive
  dependency (key → C → D, where C itself isn't part of the
key) → ✗ 3NF VIOLATED

Highest normal form satisfied = 2NF
```

FORMULA / RULE

Worked numerical — BCNF check

Q: R(A,B,C) with FDs: A→B, B→C. Candidate key = {A} (since A→B→C by transitivity, A alone determines everything). Is R in BCNF? What about 3NF?

A:

```
BCNF check — every FD's left side must be a super key:
  A→B: A is the candidate key → OK
  B→C: is B a super key? No — B alone doesn't determine A →
this FD VIOLATES BCNF

3NF check — for every FD X→Y, X must be a super key OR Y
must be a prime attribute:
  A→B: A is a super key → OK
  B→C: B is not a super key, AND C is not part of any
candidate key (not prime)
       → this FD VIOLATES 3NF too (C transitively depends
on the key A, through B)

R is NOT in BCNF and NOT in 3NF. Since the key {A} is a
single attribute, no partial
dependency is possible → 2NF is satisfied automatically.
Highest normal form satisfied = 2NF.
```

Decomposition — lossless join test

NOTES

Splitting R into R1, R2 is **lossless** if and only if: $(R1 \cap R2) \to R1$ OR $(R1 \cap R2) \to R2$ (the common attributes must functionally determine at least one of the two pieces).

WORKED EXAMPLE

Worked numerical

Q: R(A,B,C,D), FDs: A→B, A→C, C→D. Decomposed into R1(A,B,C) and R2(C,D). Is this lossless?

A:

```
Common attribute: R1 ∩ R2 = {C}
Check: does C → R1 (i.e. C → ABC)? We only know C→D, not C→A
or C→B → NO
Check: does C → R2 (i.e. C → CD)? C→D is given, and C→C is
trivial → C→CD holds → YES

Since C → R2 holds, the decomposition IS lossless.
```

Finding candidate keys quickly

RULES & FACTS

1. Attributes that **never appear on the right-hand side** of any FD (or in no FD at all) **must be in every candidate key**.
2. Compute the closure of that core. If it contains every attribute, the core is the **only** candidate key.
3. Otherwise add one attribute at a time (then pairs, …) until the closure is everything; keep only minimal sets. Every attribute in some candidate key is **prime**; the others are **non-prime**.

FORMULA / RULE

Worked numerical

Q: R(A, B, C, D, E) with F = { A→B, BC→D, D→E }. Find the candidate keys and the highest normal form.

A: A and C never appear on a right-hand side, so both are in every key. closure(AC) = {A, C, B, D, E} = everything, so **AC is the only candidate key**; prime = {A, C}, non-prime = {B, D, E}.

```
2NF: A→B has a part of the key (A) determining non-prime B →
partial dependency → 2NF violated
1NF holds, so the highest normal form is 1NF.
```

Multiple keys: for R(A, B, C) with AB→C, C→A, C→B we get C→AB, so both **C** and **AB** are candidate keys. Every left-hand side is a superkey, so R is in BCNF.

Dependency preservation vs lossless join

NOTES

A decomposition of R into $R_1 \dots R_n$ is **dependency preserving** if the FDs that hold in the pieces (the projections of F onto each R_i) together still imply every FD of F, so each FD can be checked *within a single table* without a join. It is **lossless** if joining the pieces gives back exactly R. For a binary split into R_1 and R_2: lossless iff $(R_1 \cap R_2) \to R_1$ or $(R_1 \cap R_2) \to R_2$ (the common attributes are a key of one side).

REFERENCE TABLE

Decomposition method	Lossless?	Dependency preserving?
3NF synthesis from a minimal cover	**Always**	**Always**
BCNF decomposition	**Always**	**Not guaranteed**

FORMULA / RULE

Worked example

Q: R(C, S, Z) (city, street, zip) with F = { CS→Z, Z→C }. Candidate keys: CS and SZ. Is Z→C a BCNF violation, and what happens if we decompose on it?

A: Z is not a superkey, so Z→C violates BCNF (but R is in 3NF because C is prime). Decompose into R_1(Z, C) and R_2(S, Z). The common attribute Z is a key of R_1, so the split is **lossless**. But C and S now live in different tables, so **CS→Z cannot be enforced** without a join: the decomposition is **not dependency preserving**.

Contrast: R(A, B, C) with F = {A→B, B→C} split into (A, B) and (B, C) is both lossless (B is a key of (B, C)) and dependency preserving. Split into (A, B) and (A, C) is lossless (A is a key) but loses B→C.

Multivalued dependencies and 4NF (brief)

NOTES

X ↠ Y holds when, for each X value, the set of Y values is independent of the remaining attributes: if (x, y_1, z_1) and (x, y_2, z_2) are in the relation, so are (x, y_1, z_2) and (x, y_2, z_1). Every FD is an MVD, but not conversely. **4NF** = BCNF and, for every non-trivial MVD X ↠ Y, X is a superkey. Classic case: Course ↠ Teacher and Course ↠ Book, independently; storing both lists in one table multiplies the rows, and the fix is two tables (Course, Teacher) and (Course, Book). **5NF** goes one step further and removes join dependencies not implied by keys.

5 File Organization & Indexing (B/B+ Trees)

File organization methods

REFERENCE TABLE

Method	How records are stored
Heap (unordered)	Records inserted wherever there's space; fast inserts, slow search (must scan)
Sequential	Records sorted by a key; fast range queries, but inserts/deletes are expensive (must maintain order)
Hashed	A hash function maps a key directly to a storage location/bucket; very fast exact-match lookup, poor for range queries

Index basics

KEY POINT

Clustered index
The table's actual rows are physically stored in the SAME order as the index (only one clustered index possible per table).

KEY POINT

Non-clustered (secondary) index
A separate structure pointing to rows stored elsewhere in some other order (a table can have many of these).

KEY POINT

Dense index
One index entry for EVERY search key value in the file.

KEY POINT

Sparse index
One index entry only for SOME search key values (e.g. one per block) — smaller, but needs the data to be sorted.

B-Tree vs B+-Tree

REFERENCE TABLE

	B-Tree	B+-Tree
Where data pointers live	BOTH internal nodes and leaves hold actual record pointers	ONLY leaf nodes hold record pointers; internal nodes hold just "signpost" copies of keys to guide the search
Leaf linking	Leaves not linked together	Leaves are linked in a chain — enables fast sequential/range scans
Used in practice	Less common	The standard choice for almost every real DBMS index

B+-Tree order formulas — memorize these

KEY POINT

For a B+-tree of **order p** (maximum p pointers/children per internal node):

```
Max keys per internal node        = p - 1
Max children per internal node    = p
Min children per internal node (except root) = ⌈p/2⌉
Min keys per internal node (except root)     = ⌈p/2⌉ - 1
Max keys per leaf node             = p - 1
Min keys per leaf node (except root-as-leaf) = ⌈p/2⌉ - 1
```

(The root is special: it only needs a minimum of 2 children if internal, or 1 key if it's a leaf.)

WORKED EXAMPLE

Worked numerical — min/max number of leaf nodes

Q: A B+-tree index is built on a relation with n = 100,000 records. Each node (leaf or internal) has room for at most p = 8 pointers (so max 7 keys per leaf). Find the minimum and maximum possible number of leaf nodes.

A:

```
Max keys per leaf = p - 1 = 7          → gives the MINIMUM
number of leaves (tree as full as possible)
Min keys per leaf = ⌈p/2⌉ - 1 = 4-1 = 3 → gives the MAXIMUM
number of leaves (tree as sparse as possible)

Minimum leaves = ⌈100000 / 7⌉ = 14,286
Maximum leaves = ⌈100000 / 3⌉ = 33,334
```

This "min fill → max count, max fill → min count" logic is the standard GATE pattern for these questions — always double-check which extreme (fullest vs sparsest) the question is asking about.

FORMULA / RULE

Worked numerical — tree height

Q: Using order p = 101 (max 100 keys per node) and N = 10,000 unique search key records, find the MINIMUM height of the B+-tree (best case: every node packed as full as possible).

A:

```
Leaf level: minimum leaves needed = ⌈10000 / 100⌉ = 100
leaves

One level above the leaves: a single internal node (the
root) can point to up to
p = 101 children — since we only have 100 leaves, ALL of
them fit under ONE root node.

So the tree needs just 2 levels: [Root] → [100 leaves]
Minimum height = 2
```

Hash indexing

REFERENCE TABLE

Scheme	How it works	Weakness
Static hashing	Fixed number of buckets; bucket = h(key) mod M; overflow handled by chaining overflow blocks	Long overflow chains as the file grows; reorganising means rehashing everything
Extendible hashing	A **directory** of 2^d pointers (d = **global depth**) indexed by the first d bits of the hash; each bucket has a **local depth** d′ ≤ d	Directory can double; extra indirection
Linear hashing	Buckets split one at a time in round-robin order (no directory), regardless of which bucket overflowed	Overflow chains exist temporarily

KEY POINT

Extendible hashing rules

```
• Directory has 2^d entries; a bucket with local depth d' is pointed to by exactly 2^(d − d') entries
• Bucket overflows and d' < d  →  split just that bucket (d' + 1), NO directory doubling
• Bucket overflows and d' = d  →  DOUBLE the directory (d + 1), then split the bucket
• Great for equality lookups (about 1–2 I/Os); useless for range queries (use a B+-tree)
```

GATE FAVOURITE

GATE angle

Hash index → exact-match only. If a question mentions a **range** or **ORDER BY** on the key, the B+-tree is the right answer.

Block arithmetic — blocking factor and access cost

REFERENCE TABLE

Quantity	Formula
Blocking factor (records per block)	`bf = ⌊B / R⌋` (B = block size, R = record size, unspanned)
Data blocks for r records	`b = ⌈r / bf⌉`
Heap file search	average `b/2` blocks, worst case `b`
Sorted file, search on the sort key	`⌈log₂ b⌉` (binary search)
Index entries per block	`⌊B / (key + pointer)⌋`
Primary index (sparse, one entry per data block)	index blocks = `⌈b / entries-per-block⌉`; access = `⌈log₂ index blocks⌉ + 1`
Secondary index (dense, one entry per record)	index blocks = `⌈r / entries-per-block⌉`; access = `⌈log₂ index blocks⌉ + 1`
Multilevel index	Index the index until one block remains; access = **number of levels + 1**

FORMULA / RULE

Worked numerical

Q: r = 100,000 records of 128 bytes in 4096-byte blocks, sorted on a key. An index entry is 16 bytes (12-byte key + 4-byte pointer). Find the block accesses for (a) a heap file, (b) binary search on the file, (c) a primary index, (d) a secondary index on a non-ordering field, (e) a multilevel primary index.
A: bf = 4096/128 = 32, so b = ⌈100000/32⌉ = **3125** blocks. Entries per index block = 4096/16 = 256.

```
(a) heap, average        = 3125/2 → 1563 block accesses
(b) binary search        = ⌈log₂ 3125⌉ = 12
(c) primary index        : ⌈3125/256⌉ = 13 index blocks → ⌈log₂ 13⌉ + 1 = 4 + 1 = 5
(d) secondary index      : ⌈100000/256⌉ = 391 blocks → ⌈log₂ 391⌉ + 1 = 9 + 1 = 10
(e) multilevel primary   : 13 blocks → 1 block (2 levels) → 2 + 1 = 3
```

FORMULA / RULE

Worked numerical — B+ tree order from the block size

Q: Block = 4096 B, key = 12 B, tree pointer = 8 B, record pointer = 8 B. Find the order of an internal node and the keys per leaf.
A: Internal node holding p pointers and p − 1 keys: `p·8 + (p − 1)·12 ≤ 4096` → 20p ≤ 4108 → **p = 205** (check: 205·8 + 204·12 = 4088 ≤ 4096). A leaf with k (key, record-pointer) pairs plus one next-leaf pointer: `k·(12 + 8) + 8 ≤ 4096` → k ≤ 204.4, so **204 keys per leaf**.

6 Transactions & Concurrency Control

ACID properties

REFERENCE TABLE

Property	Guarantee
Atomicity	A transaction happens completely, or not at all — no partial updates survive a crash
Consistency	A transaction moves the database from one valid state to another valid state
Isolation	Concurrent transactions don't see each other's uncommitted, intermediate changes
Durability	Once committed, changes survive any subsequent crash

Serializability — the gold standard for correctness

NOTES

A **serial schedule** runs transactions one at a time, start to finish, with zero interleaving — always correct, but slow. An interleaved (concurrent) schedule is **conflict serializable** if it's equivalent to SOME serial schedule, checked using a **precedence graph**:

KEY POINT

Draw an edge **Ti → Tj** whenever an operation of Ti **conflicts** with, and comes before, an operation of Tj on the same data item. Two operations conflict if they're from different transactions, touch the same data item, and **at least one is a WRITE** (R-W, W-R, or W-W all conflict; R-R never conflicts).

```
If the precedence graph has a CYCLE  → NOT conflict serializable
If the precedence graph is ACYCLIC   → conflict serializable
(a topological sort gives an equivalent serial order)
```

WORKED EXAMPLE

Worked numerical

Q: Schedule S: R1(A), R2(A), W1(A), R3(B), W2(B), W3(B). Build the precedence graph and determine if S is conflict serializable.
A:

```
Conflicts on A: R2(A) occurs before W1(A) → different transactions, one is a write → edge T2 → T1

Conflicts on B: R3(B) occurs before W2(B) → edge T3 → T2
                W2(B) occurs before W3(B) → edge T2 → T3

Precedence graph edges: T2→T1,  T3→T2,  T2→T3

Check for a cycle: T2→T3→T2 forms a 2-node cycle (T2 and T3 point to each other)

Result: the graph HAS a cycle → S is NOT conflict serializable.
```

View serializability

NOTES

Two schedules are **view equivalent** if (1) each transaction reads the **initial value** of an item in both or in neither, (2) every **read-from** relation (Ti reads the value written by Tj) is the same in both, and (3) the **final write** on every item is by the same transaction. A schedule is **view serializable** if it is view equivalent to some serial schedule.

KEY POINT

Key facts

```
Conflict serializable ⊂ View serializable ⊂ All schedules
A schedule that is view- but NOT conflict-serializable must contain BLIND WRITES
(a write on an item by a transaction that did not read it first)
Testing view serializability is NP-complete (no simple precedence-graph test)
```

WORKED EXAMPLE

Worked numerical

Q: S: R1(A) W2(A) W1(A) W3(A). Is it conflict serializable? View serializable?
A:

```
Precedence edges: R1(A)→W2(A): T1→T2;  W2(A)→W1(A): T2→T1  → cycle → NOT conflict serializable
View test against serial order T1, T2, T3:
  initial read of A by T1 in both ✓ · no read-from relations in either ✓ · final write on A by T3 in both ✓
→ view equivalent to T1 T2 T3 → VIEW SERIALIZABLE (W2 and W3 are blind writes)
```

Recoverability of schedules

NOTES

Serializability says nothing about what happens when a transaction **aborts**. Recoverability classes handle that:

REFERENCE TABLE

Class	Rule
Recoverable	If Tj **reads from** Ti, then Ti must **commit before** Tj commits
Cascadeless (ACA)	Tj may read an item only **after** the transaction that last wrote it has committed — no dirty reads, so no cascading aborts
Strict	No transaction may **read or overwrite** an item written by Ti until Ti commits or aborts

MUST REMEMBER

```
Strict ⊂ Cascadeless ⊂ Recoverable ⊂ All schedules
Serial schedules are strict · Strict 2PL schedules are strict + conflict serializable
Serializability and recoverability are INDEPENDENT properties (a schedule can have either, both or neither)
```

WORKED EXAMPLE

Worked numerical — classify

```
S1: W1(A) R2(A) C2 C1   T2 read from T1 but committed first → NOT recoverable
S2: W1(A) R2(A) C1 C2   dirty read, but commit order is right       → recoverable, NOT cascadeless
S3: W1(A) W2(A) C1 C2   no reads; T2 overwrites uncommitted A       → cascadeless, NOT strict
S4: W1(A) C1 R2(A) C2   reads/writes only after commit → strict
```

Two-Phase Locking (2PL)

KEY POINT

Every transaction has a **growing phase** (can only ACQUIRE locks, never release) followed by a **shrinking phase** (can only RELEASE locks, never acquire any new ones). The moment a transaction releases its first lock, it may never request another. 2PL guarantees conflict serializability.

REFERENCE TABLE

Variant	Extra rule	Benefit
Basic 2PL	Just the growing/shrinking rule above	Serializable, but can still allow cascading rollbacks
Strict 2PL	ALL locks (especially write locks) held until the transaction commits or aborts	Prevents cascading rollbacks — the industry-standard choice
Rigorous 2PL	Every lock (read AND write) held until commit/abort	Simplifies recovery further; transactions serialize in commit order

Deadlocks — detection and prevention

KEY POINT

Detection: keep a **wait-for graph** (node per transaction; edge Ti → Tj when Ti waits for a lock held by Tj). A **cycle** ⇔ **deadlock**; abort one victim on the cycle. **Prevention** uses transaction timestamps (smaller TS = **older**) when a lock conflict occurs:

REFERENCE TABLE

Scheme	Older requests a lock held by younger	Younger requests a lock held by older
Wait-Die (non-preemptive)	Older **waits**	Younger **dies** (rolled back)
Wound-Wait (preemptive)	Older **wounds** the younger (younger rolled back)	Younger **waits**

COMMON TRAP

Memory trick

In **Wait-Die the OLD wait**; in **Wound-Wait the YOUNG wait**. A restarted transaction keeps its **original timestamp**, so it eventually becomes the oldest — no starvation. Both schemes are deadlock-free (all waits go in one direction of age).

Timestamp Ordering Protocol

NOTES

Every transaction gets a unique timestamp TS(T) when it starts. Each data item tracks the timestamp of its most recent successful read (R-timestamp) and write (W-timestamp). A read or write is **rejected (and the transaction rolled back)** if it would violate the timestamp order — e.g. a transaction tries to write a value that a "younger" (later-timestamped) transaction has already read, which would make that earlier read see a "future" value it shouldn't have.

GATE FAVOURITE

GATE angle

Unlike 2PL (which can cause transactions to WAIT for locks), timestamp ordering never makes a transaction wait — it either proceeds immediately or is rolled back and restarted with a NEW timestamp. This trades lock-waiting for a different cost: rollback/restart overhead.

KEY POINT

Basic TO rules (item Q has R-TS(Q) and W-TS(Q)):

```
Read(Q) by T:   if TS(T) < W-TS(Q)  → reject, roll back T
                else execute; R-TS(Q) = max(R-TS(Q), TS(T))
Write(Q) by T:  if TS(T) < R-TS(Q)  → reject, roll back T (a younger txn already read the old value)
                if TS(T) < W-TS(Q)  → reject, roll back T (basic TO)  /  IGNORE the write (Thomas)
                else execute; W-TS(Q) = TS(T)
```

Thomas write rule: in the case TS(T) < W-TS(Q) the write is *obsolete* (a later write has already overwritten it), so instead of aborting T we simply **skip it**. This admits schedules that are **view serializable but not conflict serializable**. Basic TO guarantees conflict serializability and freedom from deadlock, but not recoverability/cascadelessness on its own.

Multiple-granularity locking & intention locks

NOTES

Data items form a hierarchy (database → file → page → record). Locking a coarse node implicitly locks everything below it; **intention locks** on ancestors announce that finer locks are held below.

REFERENCE TABLE

Held ↓ / Requested →	IS	IX	S	SIX	X
IS	✔	✔	✔	✔	✘
IX	✔	✔	✘	✘	✘
S	✔	✘	✔	✘	✘
SIX	✔	✘	✘	✘	✘
X	✘	✘	✘	✘	✘

KEY POINT

```
To lock a node in S or IS → must hold IS or IX on its parent
To lock a node in X, IX or SIX → must hold IX or SIX on its parent
Lock from the ROOT downward; release from the LEAF upward
SIX = S on the whole subtree + IX (read everything, update a few records)
```

Isolation levels and the anomalies they allow

REFERENCE TABLE

Level (SQL standard)	Dirty read	Non-repeatable read	Phantom
Read Uncommitted	possible	possible	possible
Read Committed	**prevented**	possible	possible
Repeatable Read	prevented	**prevented**	possible
Serializable	prevented	prevented	**prevented**

NOTES

Dirty read: reading another transaction's uncommitted write. **Non-repeatable read:** reading the same row twice and getting different values because another transaction committed an update in between. **Phantom:** re-running a range query and seeing new rows that another transaction inserted.

Lock modes and compatibility

REFERENCE TABLE

Held ↓ / Requested →	Shared (S)	Exclusive (X)
Shared (S)	granted	must wait
Exclusive (X)	must wait	must wait

NOTES

Many readers can share an item, but a writer needs it alone. A transaction may **upgrade** S→X (only if it is the sole holder) and, in some schemes, downgrade X→S. The **lock point** is the moment a 2PL transaction holds all its locks, and ordering transactions by their lock points gives an equivalent serial order. Basic 2PL guarantees conflict serializability but **not** freedom from deadlock and **not** recoverability (a transaction may release a lock on dirty data); strict 2PL, which holds X locks until commit, fixes the second.

Recovery — write-ahead logging

RULES & FACTS

- **Log record** `<T, X, old, new>` is written to stable storage **before** the changed page reaches disk (**write-ahead logging**). A transaction counts as committed once its `<T commit>` record is on disk.
- **Deferred modification** writes to the database only after commit, so recovery needs only **REDO**. **Immediate modification** may write before commit, so recovery needs **UNDO** (and REDO).
- **Checkpoint:** flush logs and dirty pages and note the active transactions, so recovery never scans the log before it.
- **Recovery after a crash:** REDO transactions that committed after the last checkpoint; UNDO transactions that were active (started, no commit) at the crash.

WORKED EXAMPLE

Worked example

Q: Log: <T1 start>, <T1, A, 10, 20>, <T1 commit>, <checkpoint>, <T2 start>, <T2, B, 5, 8>, <T3 start>, <T3, C, 1, 2>, <T3 commit>, then a crash. What does recovery do?

A: T1 committed **before** the checkpoint, so its updates are already on disk: nothing to do. T3 committed after the checkpoint: **REDO** it (C = 2). T2 never committed: **UNDO** it (B back to 5, its old value).

FORMULA / RULE

Shadow paging keeps a current page table and a shadow page table; updates go to new pages (copy-on-write), commit atomically swaps the table pointer, and abort just discards the new pages, so no log is needed but data gets fragmented. **ARIES** uses write-ahead logging with log sequence numbers (LSNs) and recovers in three passes: **analysis** (find dirty pages and active transactions), **redo** (repeat history from the log, even for transactions that will be undone) and **undo** (roll back the losers).

7 Problem → Standard Method Playbook

METHOD

“for every / all” query

Recognize: an entity must be related to *every* member of another relation.

```
1. Project the required entity-key pairs.
2. Use relational division R ÷ S, or
3. rewrite as NOT EXISTS (missing required match).
4. Check that duplicates cannot change the logical result.
```

Trap: ordinary join + GROUP BY needs a count equal to the number of required values; forgetting this turns “all” into “at least one”.

METHOD

Minimal / canonical cover

```
1. Split every RHS: X → YZ becomes X → Y, X → Z.
2. Minimize each LHS: remove an attribute if the reduced
determinant still implies the RHS.
3. Remove redundant FDs: temporarily delete one FD and test
whether it follows from the rest using attribute closure.
4. Recombine equivalent RHSs only at the end.
```

Trap: do not delete an FD merely because its RHS appears elsewhere; test implication.

METHOD

Schedule → conflict serializability

```
1. Keep only conflicting pairs: same item, different
transactions, at least one WRITE.
2. Draw Ti → Tj when Ti's operation occurs first.
3. Acyclic precedence graph ⇒ conflict serializable.
4. A cycle ⇒ not conflict serializable.
5. If needed, topologically order the acyclic graph to
obtain the equivalent serial order.
```

METHOD

View serializability / Thomas write rule

```
Track: initial reads, reads-from relationships, and final
writer of each item.
Thomas write rule: if TS(T) < W-TS(X), T's write is obsolete
→ ignore it rather than aborting T.
Use this when the schedule is not conflict-serializable but
may still be view-serializable.
```

METHOD

Locking / deadlock prevention

2PL: growing phase acquires locks; shrinking phase releases them. Strict 2PL keeps write locks until commit/abort.

Wait-die: older waits; younger aborts. **Wound-wait:** older preempts/aborts younger; younger waits for older.

Intention locks: use IS/IX/SIX on ancestors before finer-grained locks below.

METHOD

Index / access-path choice

```
Equality on a hashable key → hash index is attractive.
Range / ORDER BY / predecessor-successor → B+ tree.
B+ tree search: follow separator keys to leaf; linked leaves
support ordered/range scans.
Always distinguish logical comparison count from disk
I/O/page accesses.
```

1. Relational algebra division

METHOD NOTES

R(Student, Course) = {(A,DBMS),(A,CN),(B,DBMS),(C,DBMS),(C,CN)}. Required(Course) = {DBMS,CN}. Find the students who took **all** required courses.

Student A has both required pairs; B lacks CN; C has both. Therefore the result is **{A,C}**.

SQL equivalent: select students for which `NOT EXISTS` a required course missing from R.

2. Minimal cover

METHOD NOTES

Given F = {A→BC, B→C, A→B, AB→C}. Find a minimal cover.

Step 1: split A→BC into A→B and A→C.

Step 2: AB→C has extraneous B because A→C is already implied by the other dependencies.

Step 3: A→C is redundant because A→B and B→C imply A→C.

One minimal cover is **{A→B, B→C}**.

3. Precedence graph

METHOD NOTES

Schedule: $r_1(X)$, $w_1(X)$, $r_2(X)$, $w_2(X)$, $r_1(Y)$, $w_2(Y)$. What does the conflict graph imply?

On X, T_1's write precedes T_2's read/write ⇒ $T_1 \to T_2$. On Y, T_1 reads before T_2 writes ⇒ $T_1 \to T_2$. The graph is acyclic, so the schedule is **conflict serializable**, equivalent to serial order **T_1 then T_2**.

Trap: r-r pairs do not create edges.

4. Wait-die vs wound-wait

METHOD NOTES

T_1 is older than T_2. T_1 requests a lock held by T_2. What happens?

Wait-die: older T_1 may **wait** for younger T_2. **Wound-wait:** older T_1 **wounds** T_2, so T_2 is rolled back and T_1 obtains the lock.

Memory rule: wait-die = old waits; wound-wait = old wounds.

5. B+ tree vs hash

METHOD NOTES

A query asks for all employee IDs in the range 5000–7000. Which index is structurally suited?

B+ tree. After reaching the first qualifying leaf, linked leaf pages can be scanned in key order. A hash index is optimized for equality lookup and does not preserve key order for efficient range scans.

⚡ DBMS Last-Minute Recall

METHOD

- **Division = ALL**; join alone normally means “at least one”.
- Minimal cover: **split RHS → minimize LHS → remove redundant FDs**.
- Precedence graph: edge for same item + different transactions + at least one write.
- Acyclic precedence graph ⇔ conflict serializable.
- Thomas write rule can ignore obsolete writes; it can admit view-serializable schedules that are not conflict-serializable.
- **Wait-die:** older waits. **Wound-wait:** older aborts younger.
- B+ tree → ordered/range access; hash → equality access.

8 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- **Intention locks:** IS/IX on ancestors; SIX = S + IX; X conflicts with everything; IX compatible only with IS and IX.
- Selection filters rows; projection selects columns. Projection can remove duplicate tuples in the relational model.
- 2NF: 1NF + no non-prime attribute partially dependent on a candidate key. 3NF: for every non-trivial FD $X \to A$, X is a superkey or A is prime.
- B+ tree height and fan-out depend on node capacity, key/pointer sizes, and root/leaf structure; use the exact convention given in a question.
- Blocks $b = \lceil r/bf \rceil$ with $bf = \lfloor B/R \rfloor$; sorted-file search $\lceil \log_2 b \rceil$; primary index $\lceil \log_2 \text{index blocks} \rceil + 1$; multilevel index = levels + 1. B+ internal order p: $p \cdot P + (p-1) \cdot K \le B$.

LAST-MINUTE · MUST REMEMBER

- A candidate key is a minimal superkey; a primary key is one selected candidate key.
- Natural join matches attributes with the same names; be precise about join attributes and duplicate-column handling.
- WHERE filters rows before grouping; HAVING filters groups after GROUP BY.
- Functional dependency $X \to Y$ means equal X values imply equal Y values; it is a constraint on relation instances.
- B+ tree data pointers/records are stored at leaves; internal nodes primarily guide search. Leaf nodes are linked for ordered/range access.
- Conflict serializability can be tested using a precedence graph: acyclic iff the schedule is conflict-serializable.
- Two-phase locking has growing and shrinking phases; strict 2PL holds exclusive locks until commit/abort.
- **Candidate keys:** attributes never on any right-hand side are in every key; take the closure of that core. Prime attributes belong to some candidate key.
- **Isolation levels:** Read Committed stops dirty reads; Repeatable Read also stops non-repeatable reads; only Serializable stops phantoms.

LAST-MINUTE · COMMON TRAPS

- Serializability and recoverability are independent — a schedule can be one without the other.
- Hash index = equality only; range queries need a B+-tree. Extendible hashing: entries to a bucket = $2^{\text{global} - \text{local}}$.
- SQL NULL is not equal to anything, including another NULL; comparisons involving NULL use UNKNOWN.
- Recoverability, cascadelessness, strictness, and serializability are different properties—do not treat them as synonyms.
- BCNF decomposition is always lossless but may lose dependencies; 3NF synthesis from a minimal cover is always lossless and dependency preserving. Binary split lossless iff the common attributes are a key of one side.
- Basic 2PL gives conflict serializability but not deadlock freedom or recoverability; strict 2PL holds X locks to commit. 4NF removes non-trivial multivalued dependencies whose left side is not a superkey.

LAST-MINUTE · GATE FAVOURITES

- **Containment:** Strict $\subset$ Cascadeless $\subset$ Recoverable; Conflict-serializable $\subset$ View-serializable.
- BCNF requires every non-trivial FD determinant to be a superkey; BCNF is stricter than 3NF.